

ESP32 Weather & Flood Monitoring System

Hardware Wiring Guide & Complete Firmware Implementation Reference

Md. Najmus Shakib || RUET_EEE_22 || nshakib22ruet@gmail.com

1. Pin Configuration & Wiring Schematics

This section outlines the physical peripheral connections mapped to the ESP32 microcontroller for the weather station and water level tracking platform.

1.1 OLED Display Connection (SH1106 I2C)

OLED Pin	ESP32 Pin	Description
VCC	3.3V	Power supply input for the organic LED panel.
GND	GND	Common system ground.
SDA	GPIO 21	I2C Serial Data line.
SCL	GPIO 22	I2C Serial Clock line.

1.2 Climate Sensor Connection (DHT11)

DHT11 Pin	ESP32 Pin	Description
VCC	3.3V or 5V	Power supply input (Consult specific breakout specification).
DATA / OUT	GPIO 4	Single-wire digital data signal input line.
GND	GND	Common system ground.

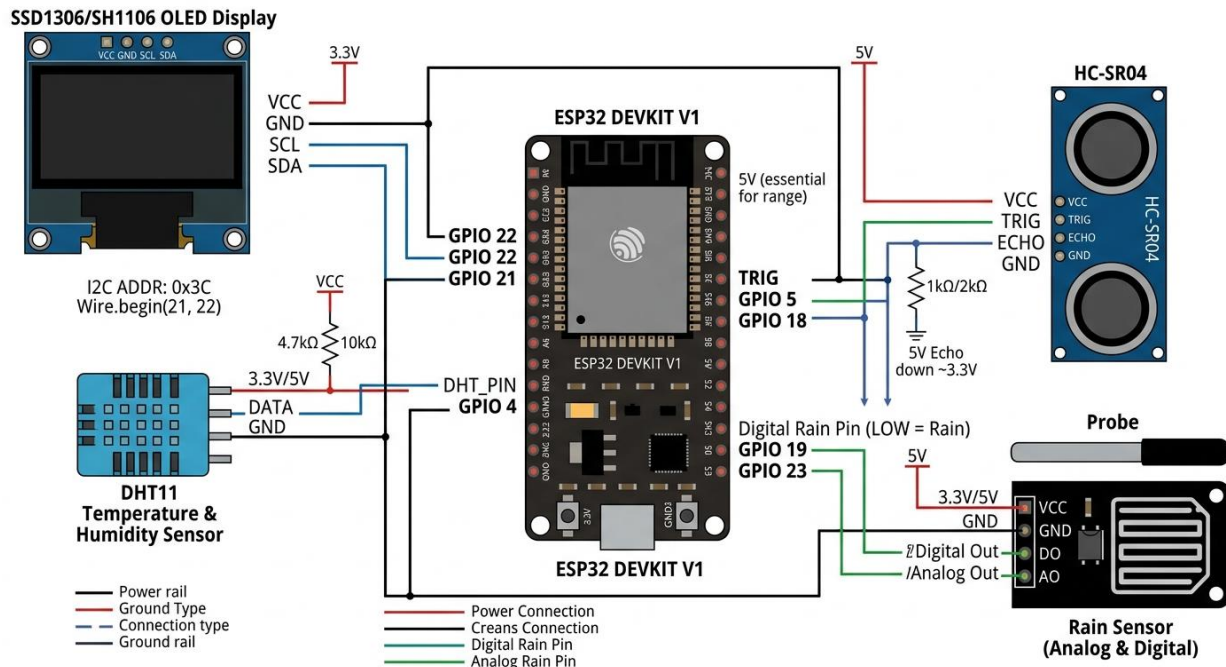
1.3 Ultrasonic Distance Sensor Connection (HC-SR04)

HC-SR04 Pin	ESP32 Pin	Description
VCC	5V	Requires stable 5V input to drive acoustic transducer pulses.
TRIG	GPIO 5	Trigger pulse output sent from the ESP32 microcontroller.
ECHO	GPIO 18	Echo signal input back to the ESP32 (Requires voltage shifting).
GND	GND	Common system ground.

1.4 Rain Sensor Module Connection

Rain Board Pin	ESP32 Pin	Description
VCC	3.3V or 5V	Power supply input for the operational amplifier interface board.
GND	GND	Common system ground.
DO (Digital Out)	GPIO 19	Digital interrupt signal line; switches LOW upon crossing rain threshold.
AO (Analog Out)	GPIO 23	Variable voltage output linked directly to an internal ESP32 ADC channel (0-4095).

2.Connection Diagram :



3. Core Firmware Implementation

The complete, optimized code block provided below handles BLE profile configuration, peripheral pooling, ultrasonic noise filtration/calibration, and formats the output buffer matching your remote application parsing layers.

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <DHT.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>

// ===== OLED =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_ADDR 0x3C

Adafruit_SH1106G display = Adafruit_SH1106G(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== PINS =====
```

```

const int DHT_PIN   = 4;
const int TRIG      = 5;
const int ECHO      = 18;
const int RAIN_D    = 19;    // digital rain pin (LOW = rain)
const int RAIN_A    = 23;    // analog rain pin

#define DHTTYPE DHT11
DHT dht(DHT_PIN, DHTTYPE);

// ===== BLE =====
// MUST match Flood.html exactly
#define SERVICE_UUID "facade00-babe-cafe-dead-beefcafebab"
#define CHAR_UUID    "facade01-babe-cafe-dead-beefcafebab"

BLECharacteristic *pChar;
bool connected = false;

// ===== CALIBRATION =====
// HTML uses a rolling average of first 5 readings for base.
// We mirror that: collect up to CALIB_SAMPLES distances,
// then lock the median as baseDist and set calib = true.
#define CALIB_SAMPLES 5
float calibBuf[CALIB_SAMPLES];
int calibCount = 0;
float baseDist = -1;
bool calibrated = false;

// ===== FLOOD STREAK =====
// HTML reads parts[8] as 'streak' (consecutive high-water readings).
// We count it here and send it so the HTML can use it.
int floodStreak = 0;
unsigned long highWaterStart = 0;

// ===== BLE CALLBACKS =====
class Callbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* p) { connected = true; }
    void onDisconnect(BLEServer* p) {
        connected = false;
        BLEDevice::startAdvertising();
    }
};

// ===== ULTRASONIC =====
float readDist() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    long t = pulseIn(ECHO, HIGH, 30000);
    if (t == 0) return -1;

```

```

    return (t * 0.0343f) / 2.0f;
}

// ===== OLED =====
void updateOLED(float t, float h, float dist, float baseDis,
               float waterPct, bool rain, bool flood, bool calib) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);

    display.setCursor(0, 0);
    display.print("Temp: ");
    if (isnan(t)) display.println("ERR");
    else { display.print(t, 1); display.println(" C"); }

    display.print("Hum : ");
    display.print(h, 1); display.println(" %");

    display.print("Dist: ");
    if (dist < 0) display.println("ERR");
    else { display.print(dist, 1); display.println(" cm"); }

    display.print("Base: ");
    if (calib) { display.print(baseDis, 1); display.println(" cm"); }
    else display.println("Calibrating...");

    display.print("Water: "); display.print(waterPct, 1); display.println(" %");
    display.print("Rain: "); display.println(rain ? "YES" : "NO");
    display.print("Flood: "); display.println(flood ? "DANGER!" : "SAFE");

    display.display();
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    pinMode(RAIN_D, INPUT);

    dht.begin();

    // OLED init
    Wire.begin(21, 22);
    if (!display.begin(OLED_ADDR, true)) {
        Serial.println("SH1106 init failed");
        for (;;)
    }
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);

```

```

display.setCursor(0, 0);
display.println("Starting...");
display.display();
delay(1500);

// BLE init - device name must match HTML filter: 'ESP32_WeatherFlood'
BLEDevice::init("ESP32_WeatherFlood");
BLEServer *server = BLEDevice::createServer();
server->setCallbacks(new Callbacks());

BLEService *service = server->createService(SERVICE_UUID);
pChar = service->createCharacteristic(CHAR_UUID,
                                     BLECharacteristic::PROPERTY_NOTIFY);
pChar->addDescriptor(new BLE2902());
service->start();
BLEDevice::startAdvertising();

Serial.println("BLE advertising as ESP32_WeatherFlood");
}

// ===== LOOP =====
void loop() {
    // --- Read DHT11 ---
    float temp = dht.readTemperature();
    float hum = dht.readHumidity();

    if (isnan(temp)) temp = 0;
    if (isnan(hum)) hum = 0;
    else hum = hum * 0.86f + 2.0f;

    // --- Read Ultrasonic ---
    float dist = readDist();

    // --- Read Rain sensor ---
    bool isRain = (digitalRead(RAIN_D) == LOW);
    int rainADC = analogRead(RAIN_A);

    // --- Calibration ---
    if (!calibrated && dist > 0) {
        if (calibCount < CALIB_SAMPLES) {
            calibBuf[calibCount++] = dist;
            Serial.println("Calib sample " + String(calibCount) + ": " + String(dist));
        }
        if (calibCount >= CALIB_SAMPLES) {
            float sum = 0;
            for (int i = 0; i < CALIB_SAMPLES; i++) sum += calibBuf[i];
            baseDist = sum / CALIB_SAMPLES;
            calibrated = true;
            Serial.println("BASE SET: " + String(baseDist));
        }
    }
}

```

```

// --- Water level calculation ---
const float TANK_DEPTH_CM = 35.0f;
float rise      = 0;
float waterPct  = 0;
bool  flood     = false;

if (calibrated && dist > 0) {
    rise = baseDist - dist;
    if (rise < 0) rise = 0;

    waterPct = (rise / TANK_DEPTH_CM) * 100.0f;
    if (waterPct > 100) waterPct = 100;

    if (waterPct > 80 && isRain) {
        if (highWaterStart == 0) highWaterStart = millis();
        if (millis() - highWaterStart > 20000) {
            flood = true;
            floodStreak++;
        }
    } else {
        highWaterStart = 0;
        floodStreak    = 0;
        flood           = false;
    }
}

// --- OLED update ---
updateOLED(temp, hum, dist, baseDist, waterPct, isRain, flood, calibrated);

// --- BLE notify ---
if (connected) {
    String data = "WX:"
        + String(temp,    1) + ","
        + String(hum,     1) + ","
        + String(dist < 0 ? 0 : dist, 1) + ","
        + String(baseDist < 0 ? 0 : baseDist, 1) + ","
        + String(isRain   ? 1 : 0) + ","
        + String(rainADC)          + ","
        + String(calibrated ? 1 : 0) + ","
        + String(flood     ? 1 : 0) + ","
        + String(floodStreak);

    pChar->setValue(data.c_str());
    pChar->notify();
    Serial.println("Sent: " + data);
}

delay(1000);
}

```